Code Plagiarism Checker - System Design Report

Group 8, SDN260S Software Design II, CPUT · Nov 1, 2026 · Dembe Kwinda

1. Problem definition

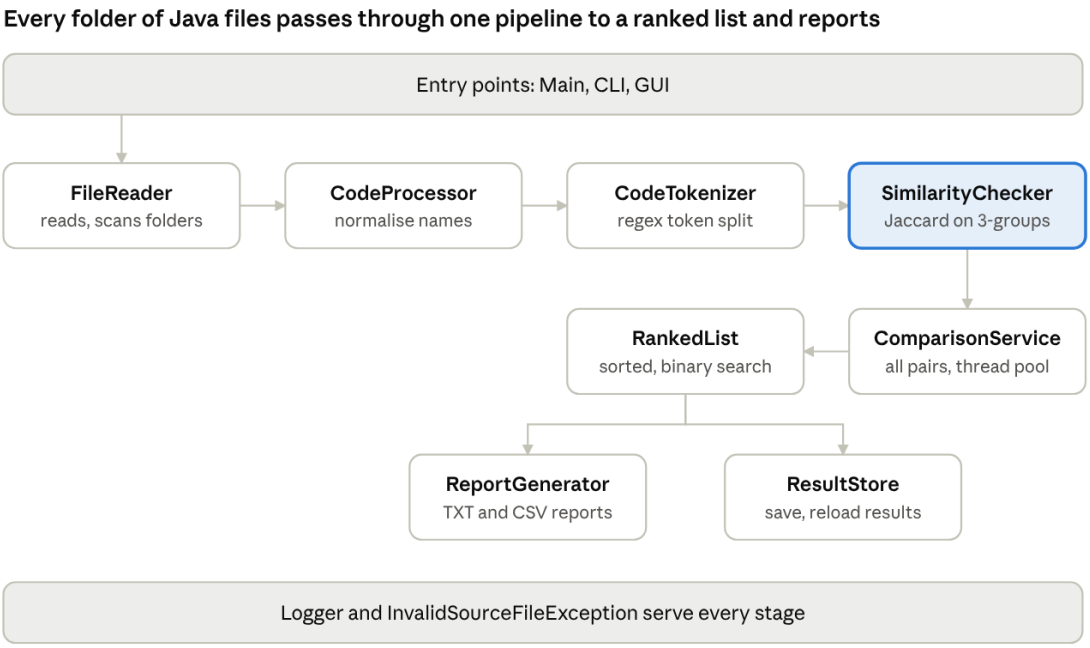
Students can copy a Java assignment and rename the variables, which defeats a plain text comparison. Our tool compares every pair of submitted `.java` files on their code structure instead of their wording, ranks the pairs by similarity, and flags pairs at or above 70% for a person to review.

The main users are lecturers and markers who must check a folder of submissions quickly and fairly. A flag only marks a pair for human review; it is not a finding of plagiarism. The group approved the similarity method and the 70% threshold on 1 Nov 2026.

The tool runs offline, reads only the files it is given, and offers three interfaces: a console menu (`--cli`), a Swing GUI (`--gui`) and a one-shot run (`Main <path>`).

2. System design

The system is 18 classes in one package, `plagiarismchecker`. The pipeline below is the core; `CodeFile`, `SimilarityResult` and `Pair` carry data between stages, and `ResultsTableModel` feeds the GUI table.



system pipeline · 8 classes, 1 parallel step

Read left to right: a file is read, normalised, tokenised and scored; `ComparisonService` repeats that for every pair in parallel, `RankedList` sorts the scores, and the results go to a report or the store. The UML class diagram is in `docs/uml_class_diagram.png`.

3. Design decisions

The Check-In 1 design held up; five changes were made while building. Each class has one job, so a change to one stage (for example the tokenizer) does not touch the others.

Check-In 1 design	What the code does now	Why
`CodeProcessor.tokenise()`	Moved to a new `CodeTokenizer` class	Cleaning text and splitting tokens are separate jobs
Comparison inside the checker	New `ComparisonService`	One place to run all pairs, in parallel
Plain lists of results	New `Pair<A,B>` and `RankedList<T>`	Reusable generic types; sorted insert and binary search
Results lost on exit	New `ResultStore` (serialization)	Reload earlier runs; uses an allow-list input filter
Print statements	New `Logger` and `InvalidSourceFileException`	Proper logging and clear errors for bad files

Two choices are worth defending in the demo. Renamed variables are mapped to a single `ID` token, so renaming does not hide a copy. The report always says "flagged for review" because a similarity score cannot prove intent.

4. Data structures and algorithms

Similarity is computed in four steps, and the whole run is $O(n^2)$ in the number of files.

1. **Normalise** (`CodeProcessor`): strip comments and `package`/`import` lines; strings become `STR`, numbers `NUM`, other names `ID`; keywords are kept.
2. **Tokenise** (`CodeTokenizer`): split with regular expressions into a token list.
3. **Group**: overlapping 3-token groups form a set per file.
4. **Score** (`SimilarityChecker`): Jaccard similarity, the size of the intersection of two sets divided by the size of their union. A pair at or above 70% is flagged.

`J(A,B) = |A ∩ B| / |A ∪ B|`

`FileReader.scanRecursively` walks folders by recursion. `RankedList<T>` keeps results sorted on insert and uses binary search, so a lookup costs O(log n). Comparing n files means n(n-1)/2 pairs: 14 sample files give 91 pairs and 160 files give 12,720. Doubling the files roughly quadruples the time, which matches the benchmark (about 4× per doubling). The full analysis is in `docs/BIG_O.md`.

5. Advanced features

Four advanced topics from the course are used in the working code, not just described.

Topic	Where	What it does
Generics	`Pair<A,B>`, `RankedList<T>`	Type-safe pair and sorted list reused across the program
Serialization	`ResultStore`	Saves and reloads results; an `ObjectInputFilter` allow-list rejects unexpected classes
Multithreading	`ComparisonService` (`ExecutorService`), GUI (`SwingWorker`)	Compares pairs in parallel; the GUI stays responsive while it runs
Streams	Light use in sorting and filtering results	Short, readable result handling

On our 2-core test machine, 160 files (12,720 pairs) took 5,518 ms on one thread and 3,109 ms on two, a speed-up of about 1.77×. These timings come from a test machine with few processor cores, so the speed-up on a laptop with more cores will differ.

6. Testing strategy and evidence

All 87 automated tests pass (18 test classes, `mvn test`: 0 failures), run on two team members' machines; Bandile Qwane's run on 1 Nov 2026 finished with 87 run, 0 failures, 0 errors, 0 skipped, BUILD SUCCESS in 40.3 s. The tests cover each class on its own, plus an end-to-end run over the sample folder.

ID	Case	Type	Input	Result
N1	Exact copy	Normal	Alice and Bob	100%, flagged
N2	Renamed variables	Normal	Gina and Hank	100%, flagged
N3	Partly changed copy	Normal	Dan and Eve	80.4%, flagged
N4	Similar, below threshold	Normal	StudentC and Frank	66.7%, not flagged
N5	Partial copy (two methods) in a different program	Normal	Gina and Kim	61.0%, not flagged (borderline, worth a look)
N6	Whole sample set	Normal	14 files	91 pairs, 4 flagged
E1	Empty file	Edge	One empty `.java` file	Warning "File is empty", nothing loaded, exit code 1
E2	One file only	Edge	One valid file	"Add at least two files to compare

ID	Case	Type	Input	Result
				them"
E3	Identical files	Edge	A.java and B.java, same code	100.0%, flagged
E4	Wrong path	Error	`/no/such/folder`	"File does not exist", exit code 1, no crash
E5	Not a Java file	Error	A `.txt` file	"Only .java files can be checked", exit code 1
E6	No arguments	Error	None	Usage message, exit code 2

The full run of the demo folder is reproduced by `Main <path>`; the TXT and CSV reports are the sample outputs. Cases E1 to E6 were run on the integrated project on 1 Nov 2026. Murunwa Khorommbi also confirmed a clean build, the Maven tests, the CLI, sample comparisons, a wrong path and a one-file input on her computer; her screenshots are in her Check.docx. Kagiso Lengolo's separate edge-case harness (8 of 8 passed, including the 69.9%, 70.0% and 80.0% threshold cases) tests his own prototype, which uses a different method (longest common subsequence), so it is not counted as evidence for this project. The largest tests were a pair of 1,300-line files and a generated 160-file folder.

7. GA2 reflection: application of scientific and engineering knowledge

We applied set theory and algorithm analysis to a practical problem. Jaccard similarity on 3-token groups turns "do these two programs look alike" into a number between 0 and 100%, and normalising names to `ID` removes the easiest way to disguise a copy.

We used Big O to predict cost before measuring it: n files mean n(n-1)/2 comparisons, so time grows with the square of the folder size. Our benchmark confirmed this, and we added threads to use the work that can run in parallel.

What we learned: a design that looked complete at Check-In 1 still needed five changes once we tested it. Measuring and testing, not assuming, decided each one.

8. GA7 reflection: sustainability and impact of engineering activity

A wrong flag can harm a student, so the tool reports "flagged for review", never "plagiarised". The 70% threshold was approved by the group on 1 Nov 2026, and a person makes the final decision.

The tool runs offline on the user's own machine, so student code is not uploaded to any service. Saved results are loaded through an allow-list filter so a tampered file cannot run unexpected code. It needs no special hardware, and its $O(n^2)$ cost is documented so users know that very large folders take longer.

Limits we accept: similar code can be honest (short exercises, shared templates), so scores are a starting point for a conversation, not evidence on their own.

9. Team collaboration and process

Group 8 has six members: Dembe Kwinda, Murunwa Grace Khorommbi, Kagiso Lengolo, Liphelo Tayo, Ndugiselo Masase and Bandile Qwane. Roles are those on the group hub; the last column is what each person does for Check-In 3 (due 1 Nov 2026, 23:59). Task status is tracked on the group hub.

Member	Role	Check-In 3 task (31 Oct)
Dembe Kwinda	Project coordination and system design	Integrate, final demo run, design report, submit
Murunwa Grace Khorommbi	Java implementation and core classes	Run the project clean from the zip; fix any core-class bug found
Kagiso Lengolo	Similarity algorithm and data structures	Write the edge-case tests (empty file, single file, identical files); check the 70% threshold on the sample set
Liphelo Tayo	GUI and CLI development	Screenshot GUI results and CLI menu; check error messages on a bad path
Ndugiselo Masase	Testing and documentation	README and user guide; collect test evidence and sample outputs
Bandile Qwane	Code development and testing	Run `mvn test` and report the result; test the project on a second computer

Version control evidence (GitHub commit history and a coordination log from the group hub) is to be attached.

10. Limitations and next steps

The tool compares structure, so it can miss copies that reorder statements heavily and can flag honest code that follows a shared template. Cost grows with the square of the folder size.

- ☐ Group approves the similarity method and the 70% threshold
- ☐ Add empty-file, single-file and large-folder test cases
- ☐ Record benchmark timings from a team member's own machine in `docs/BIG_O.md`
- ☐ Fill in roles and contributions; attach GitHub history
- ☐ Final submission on 8 Nov 2026: zipped source, ReadMe, this report, reflection, demo